

# Corrupted by Thanos's Attack

MIRAGE 2.0 — Forensics — Analysis Status Report (Unresolved)

## CATEGORY

Forensics

## FILES

thanos\_firmware\_dump.bin (two variants)

## DEPLOYMENT

Offline analysis — no execution required

## VARIANTS

Corrupted\_by\_Thanos\_Attack.zip,  
thanos\_keks.zip

## STATUS

UNRESOLVED — flag not recovered

**NOTE:** This document is an honest status report, not a solved writeup. Despite two independent, deep analysis passes, the final flag for either variant of this challenge was **not** recovered. It is included here so the organizer can compare it against their own reference solution and either patch the challenge or point out what was missed.

## Challenge Premise

Both zips ship a raw `thanos_firmware_dump.bin` under a title of “Corrupted by Thanos's Attack”, describing a damaged firmware image whose recovery archive must be located and decoded. The `thanos_keks.zip` variant additionally warns: “Some strings look like flags. Do not trust a flag merely because strings finds it.”

## Container Format (fully reverse engineered)

Both files use the same custom TLV-style container:

```
offset 0x00  magic      'THNX'          (4 bytes)
offset 0x04  version    u32 LE
offset 0x08  total_size u32 LE (== file size, in the clean variant)
offset 0x0c  label      16 bytes, NUL-padded ASCII
offset 0x1c  num_sections u32 LE
offset 0x20  section table: N x 20-byte entries:
                name[8], offset(u32), size(u32), crc32(u32)
```

## Variant 1: Corrupted\_by\_Thanos\_Attack.zip (“clean”)

Header parses cleanly: 4 sections, BOOT, LOGS, CONFIG, RECOVERY — **all four CRC32-valid**. CONFIG contains the line `ARCHIVE=RECOVERY`, pointing at RECOVERY (202 bytes) as the payload of interest. That section's bytes are statistically indistinguishable from random data (Shannon entropy ~6.9/8) and contain no plaintext.

Techniques tried against the RECOVERY blob, all unsuccessful:

- binwalk against the whole image and the isolated blob — zero signatures detected.
- Exhaustive raw-deflate / bz2 / lzma decompression attempts at every possible starting byte offset.

- Single-byte XOR / ADD / SUB brute force, scored by printable-character ratio.
- 50+ multi-byte XOR and RC4 key candidates: the container's label, the "RECOVERY\_SALT" string seen in the other variant, section names, and MD5/SHA1/SHA256 digests of all of the above, tried forwards and reversed.
- Classical repeating-key-XOR cryptanalysis (Kasiski examination / index-of-coincidence) across key lengths 1–60.
- Bit-level transforms: nibble swap, bit reversal, atbash, high-bit stripping.
- xorshift32-style PRNG keystreams seeded from header fields (version, size, the section CRCs).
- Two-time-pad checks against the high-entropy tail bytes found in the BOOT/LOGS/CONFIG sections (each has readable ASCII header text followed by a long high-entropy tail past that text — confirmed to be genuinely random, not a simple fill pattern, but no key recovered from them either).
- Inter-section gap bytes — confirmed to be pure alignment padding, nothing hidden there.

## Variant 2: thanos\_keks.zip ("trap")

The header itself is deliberately mangled: `version` and `total_size` are garbage, and a spliced-in 20-byte string (SNAP-RECOVERY-19) sits exactly where `num_sections` and the start of the section table should be — a naive parser using the clean variant's fixed offsets breaks immediately. Manually locating section names via `strings -t x` and re-deriving a 20-byte-stride table recovers **nine** section entries, **all nine CRC32-valid** — CRC validity alone does not distinguish real data from decoys here.

Four of the nine sections (BOOTLOG, TRACE, CONFIG, CACHE) contain readable log/config text, each followed by **two** plaintext fake flags and then a single-byte padding fill (0xf2, 0x47 ('G'), 0x39 ('9') respectively) — eight fake flags in total, all confirmed CRC-valid, all the same shape (20-character lowercase alphanumeric), with no property (hash, checksum, length, position) distinguishing any one of them as more genuine than another.

The remaining five sections have short, meaningless four-character names (X7Q2, N4KP, Z9LM, R2VX, H8DS) and high-entropy contents. These are the strongest remaining lead: they resisted every XOR/keying attempt tried (individually, concatenated in several orders, and cross-keyed against the clean variant's RECOVERY blob). One structural oddity was noted but not turned into a working decode: Z9LM is 32 bytes (a SHA-256 digest's length) versus the other four sections' 36 bytes, and is the only one of the five with no 0xa5 filler gap immediately after it.

## Recommendation

Given the volume of techniques ruled out across two independent passes, this likely needs one of: (a) a hint or technique the organizer intended that neither pass considered, (b) a fix to the packaging so the intended decode path actually works (analogous to the glibc and missing-flag bugs found and fixed in other challenges in this set), or (c) an update to the challenge README with a stronger nudge toward the intended tool/technique. Recommend the organizer test this pair against their own reference solve script before the event goes live.